EDIÇÃO COMPLETA · TEORIA, MISSÕES E GABARITO

Try, Catch & Learn

< / >

Caderno de Missões & Gabarito Comentado III

Fundamentos de Programação em C · Fases 5 e 6

- Teoria completa de Vetores, Matrizes e Structs
- 13 Missões com contextualização e lembretes teóricos
- 23 Códigos C resolvidos com saídas reais e análise de erros comuns

Caderno do Aluno + Material do Professor

Bem-vindo(a) à Edição Integrada

A união do enunciado prático e da análise pedagógica comentada

Esta edição integrada reúne tudo o que o programador e o monitor precisam: o enunciado completo de cada missão (com contexto, lembretes teóricos e passos) acompanhado do gabarito oficial em C, saída real de execução e comentários pedagógicos com erros comuns.

> **Como aproveitar ao máximo:**
> Tente resolver cada missão antes de consultar o gabarito. Compare sua lógica com a solução de referência e veja se você evitou os erros comuns listados nos comentários pedagógicos.

F5.1 As Caixas Numeradas

Preenchendo e exibindo um vetor

Um armazém de logística recebeu 5 caixas e precisa anotar o peso de cada uma numa única lista, para depois conferir tudo de uma vez.

> 💡 **Lembrete teórico:** Declare o vetor com `int numeros[5];`, preencha cada posição com um `for` usando `scanf("%d", &numeros[i]);` e exiba todas com outro `for`.

Sua missão:

1. Crie um vetor `numeros[5]`.
2. Use um `for` para ler os 5 números digitados pelo usuário.
3. Use outro `for` para exibir todos os números, um ao lado do outro.

EXEMPLO DE EXECUÇÃO

```
Digite o numero 1: 10
Digite o numero 2: 25
Digite o numero 3: 3
Digite o numero 4: 47
Digite o numero 5: 8
Voce digitou: 10 25 3 47 8
```

F5.1 As Caixas Numeradas

Gabarito Oficial

CÓDIGO EM C (F5.1) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int numeros[5];
int i;
for (i = 0; i < 5; i++) {
printf("Digite o numero %d: ", i + 1);
scanf("%d", &numeros[i]);
}
printf("Voce digitou: ");
for (i = 0; i < 5; i++) {
printf("%d ", numeros[i]);
}
printf("\n");
return 0;
}
```

EXECUÇÃO REAL

```
Digite o numero 1: 10
Digite o numero 2: 25
Digite o numero 3: 3
Digite o numero 4: 47
Digite o numero 5: 8
Voce digitou: 10 25 3 47 8
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: usar o mesmo i em dois for separados sem reiniciar — como for sempre reinicia sua própria variável de controle, isso não é problema aqui, mas vale mostrar que cada for tem seu próprio ciclo de vida para i.

Try, Catch & Learn

F5.2 A Mão de Cartas

Vetor pré-preenchido e acesso por índice

Num jogo de cartas, a mão do jogador já vem com 6 cartas sorteadas. O programa precisa mostrar a mão inteira e também destacar o valor de cartas específicas, em posições escolhidas.

> 💡 **Lembrete teórico:** Um vetor pode já nascer preenchido na declaração: `int cartas[6] = {12, 45, 7, 90, 33, 61};`. Cada posição é acessada diretamente pelo índice, sem precisar de `scanf`.

Sua missão:

1. Declare o vetor `cartas[6]` já com os valores `{12, 45, 7, 90, 33, 61}`.
2. Mostre a mão inteira com um `for`.
3. Depois, mostre separadamente o valor das cartas nas posições 0, 3 e 5 (sem usar laço para essa parte).

EXEMPLO DE EXECUÇÃO

```
As cartas da mao sao: 12 45 7 90 33 61
Carta na posicao 0: 12
Carta na posicao 3: 90
Carta na posicao 5: 61
```

F5.2 A Mão de Cartas

Gabarito Oficial

CÓDIGO EM C (F5.2) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int cartas[6] = {12, 45, 7, 90, 33, 61};
int i;
printf("As cartas da mao sao: ");
for (i = 0; i < 6; i++) {
printf("%d ", cartas[i]);
}
printf("\n");
printf("Carta na posicao 0: %d\n", cartas[0]);
printf("Carta na posicao 3: %d\n", cartas[3]);
printf("Carta na posicao 5: %d\n", cartas[5]);
return 0;
}
```

EXECUÇÃO REAL

```
As cartas da mao sao: 12 45 7 90 33 61
Carta na posicao 0: 12
Carta na posicao 3: 90
Carta na posicao 5: 61
```

> 🧠 **Comentários & Ponto Pedagógico:**
> Erro comum: tentar inicializar o vetor com menos ou mais valores do que o tamanho declarado, como int cartas[6] = {12, 45, 7}; — as posições não preenchidas viram automaticamente 0, o que pode confundir se o aluno não perceber.

F5.3 O Tesouro dos Oito Baús

Vetor + acumulador

Um grupo de aventureiros encontrou uma sala com 8 baús. Cada baú tem uma quantidade diferente de moedas, e o grupo quer saber o total do tesouro.

> 💡 **Lembrete teórico:** A combinação de vetor com acumulador é igual à da Fase 3, só que agora cada valor lido vai para uma posição do vetor em vez de ser descartado: `moedas[i] = valor; total = total + moedas[i];` (ou somar direto durante a leitura, como no gabarito).

Sua missão:

1. Declare o vetor `moedas[8]`.
2. Use um `for` para ler a quantidade de moedas de cada baú e já somar ao total.
3. Mostre o total de moedas do tesouro.

EXEMPLO DE EXECUÇÃO

```
Moedas no bau 0: 10
Moedas no bau 1: 5
Moedas no bau 2: 20
Moedas no bau 3: 8
Moedas no bau 4: 0
Moedas no bau 5: 15
Moedas no bau 6: 3
Moedas no bau 7: 9
Total de moedas do tesouro: 70
```

F5.3 O Tesouro dos Oito Baús

Gabarito Oficial

CÓDIGO EM C (F5.3) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int moedas[8];
int i, total = 0;
for (i = 0; i < 8; i++) {
printf("Moedas no bau %d: ", i);
scanf("%d", &moedas[i]);
total = total + moedas[i];
}
printf("Total de moedas do tesouro: %d\n", total);
return 0;
}
```

EXECUÇÃO REAL

```
Moedas no bau 0: 10
Moedas no bau 1: 5
Moedas no bau 2: 20
Moedas no bau 3: 8
Moedas no bau 4: 0
Moedas no bau 5: 15
Moedas no bau 6: 3
Moedas no bau 7: 9
Total de moedas do tesouro: 70
```

🧠 **Comentários & Ponto Pedagógico:**
Mostre que somar moedas[i] durante a leitura (como no gabarito) ou depois, num segundo for, dá o mesmo resultado — a diferença é só estilo.

F5.4 O Recorde do Campeonato

Busca do maior valor

Seis jogadores disputaram uma partida e o sistema precisa descobrir, entre as pontuações digitadas, qual foi a maior — o recorde da rodada.

> 💡 **Lembrete teórico:** Um padrão clássico: guarde o primeiro valor do vetor numa variável `maior` e, percorrendo o resto do vetor, atualize `maior` sempre que encontrar um valor ainda maior.

Sua missão:

1. Declare o vetor `pontuacoes[6]` e leia os 6 valores.
2. Inicie `maior` com `pontuacoes[0]`.
3. Percorra o vetor a partir da posição 1, comparando cada valor com `maior` e atualizando quando necessário.
4. Mostre a maior pontuação encontrada.

EXEMPLO DE EXECUÇÃO

```
Pontuacao do jogador 1: 340
Pontuacao do jogador 2: 580
Pontuacao do jogador 3: 210
Pontuacao do jogador 4: 999
Pontuacao do jogador 5: 450
Pontuacao do jogador 6: 120
A maior pontuacao foi: 999
```

F5.4 O Recorde do Campeonato

Gabarito Oficial

CÓDIGO EM C (F5.4) GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int pontuacoes[6];
int i, maior;
for (i = 0; i < 6; i++) {
printf("Pontuacao do jogador %d: ", i + 1);
scanf("%d", &pontuacoes[i]);
}
maior = pontuacoes[0];
for (i = 1; i < 6; i++) {
if (pontuacoes[i] > maior) {
maior = pontuacoes[i];
}
}
printf("A maior pontuacao foi: %d\n", maior);
return 0;
}
```

EXECUÇÃO REAL

```
Pontuacao do jogador 1: 340
Pontuacao do jogador 2: 580
Pontuacao do jogador 3: 210
Pontuacao do jogador 4: 999
Pontuacao do jogador 5: 450
Pontuacao do jogador 6: 120
A maior pontuacao foi: 999
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: iniciar maior com 0 em vez de pontuacoes[0]. Isso quebra se todas as pontuações forem negativas, já que nenhuma superaria o 0 inicial. Sempre inicie com o primeiro elemento real do vetor.

Try, Catch & Learn

F5.5 A Semana Mais Fria

Busca do menor valor

Uma estação meteorológica registrou a temperatura de cada um dos 7 dias da semana e quer saber qual foi o dia mais frio.

> 💡 **Lembrete teórico:** O mesmo padrão da questão anterior, mas invertido: em vez de `if (valor > maior)`, usamos `if (valor < menor)`.

Sua missão:

1. Declare o vetor `termometros[7]` e leia as 7 temperaturas.
2. Encontre a menor temperatura da semana, seguindo a mesma lógica da Questão 4.
3. Mostre o resultado.

EXEMPLO DE EXECUÇÃO

```
Temperatura do dia 1: 18
Temperatura do dia 2: 15
Temperatura do dia 3: 9
Temperatura do dia 4: 22
Temperatura do dia 5: 11
Temperatura do dia 6: 14
Temperatura do dia 7: 17
A temperatura mais fria da semana foi: 9
```

F5.5 A Semana Mais Fria

Gabarito Oficial

CÓDIGO EM C (F5.5) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int termometros[7];
int i, menor;
for (i = 0; i < 7; i++) {
printf("Temperatura do dia %d: ", i + 1);
scanf("%d", &termometros[i]);
}
menor = termometros[0];
for (i = 1; i < 7; i++) {
if (termometros[i] < menor) {
menor = termometros[i];
}
}
printf("A temperatura mais fria da semana foi: %d\n", menor);
return 0;
}
```

EXECUÇÃO REAL

```
Temperatura do dia 1: 18
Temperatura do dia 2: 15
Temperatura do dia 3: 9
Temperatura do dia 4: 22
Temperatura do dia 5: 11
Temperatura do dia 6: 14
Temperatura do dia 7: 17
A temperatura mais fria da semana foi: 9
```

🧠 **Comentários & Ponto Pedagógico:**
Mesmo raciocínio da Q4, espelhado: vale pedir para o aluno explicar, com as próprias palavras, por que trocamos só o operador de comparação (> por <) e nada mais na lógica.

F5.6 A Despensa Mágica

Busca de um valor específico (com break)

Uma cozinha encantada guarda 5 ingredientes especiais, cada um identificado por um código. O cozinheiro quer saber se um ingrediente específico está disponível na despensa, sem precisar abrir todas as prateleiras manualmente.

> 💡 **Lembrete teórico:** Para procurar um valor dentro de um vetor, percorremos as posições comparando cada uma com o valor procurado. Ao encontrar, podemos usar uma "bandeirinha" (`encontrado = 1;`) e `break` para parar a busca imediatamente, sem continuar olhando o resto do vetor.

Sua missão:

1. Declare o vetor `ingredientes[5] = {7, 14, 2, 30, 9};`.
2. Leia o código do ingrediente procurado.
3. Percorra o vetor comparando cada posição com o valor procurado; se encontrar, marque `encontrado = 1` e pare a busca com `break`.
4. Mostre se o ingrediente está ou não na despensa.

EXEMPLO DE EXECUÇÃO

```
=== Execução: encontrado ===
Digite o codigo do ingrediente: 30
Ingrediente 30 esta na despensa!

=== Execução: não encontrado ===
Digite o codigo do ingrediente: 99
Ingrediente 99 NAO esta na despensa.
```

F5.6 A Despensa Mágica

Gabarito Oficial

CÓDIGO EM C (F5.6) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int ingredientes[5] = {7, 14, 2, 30, 9};
int procurado, i, encontrado = 0;
printf("Digite o codigo do ingrediente: ");
scanf("%d", &procurado);
for (i = 0; i < 5; i++) {
if (ingredientes[i] == procurado) {
encontrado = 1;
break;
}
}
if (encontrado == 1) {
printf("Ingrediente %d esta na despensa!\n", procurado);
} else {
printf("Ingrediente %d NAO esta na despensa.\n", procurado);
}
return 0;
}
```

EXECUÇÃO REAL

```
: encontrado
Digite o codigo do ingrediente: 30
Ingrediente 30 esta na despensa!

Execução: não encontrado
Digite o codigo do ingrediente: 99
Ingrediente 99 NAO esta na despensa.
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: esquecer o break, o que não muda o resultado final aqui (já que encontrado já foi marcado), mas faz o programa continuar comparando desnecessariamente até o fim do vetor — um desperdício de trabalho que fica pior quanto maior o vetor.

F5.7 A Calculadora de Médias (Revisitada)

Vetor passado para uma função

A escola quer melhorar o sistema de médias da Fase 4: em vez de uma função que recebe notas separadas uma a uma, ela deve receber o vetor inteiro de notas de uma só vez.

> 💡 **Lembrete teórico:** Uma função pode receber um vetor como parâmetro: `float calcularMedia(float notas[], int tamanho)`. Dentro da função, um `for` percorre `notas[i]` normalmente, como se o vetor fosse local.

Sua missão:

1. Crie a função `float calcularMedia(float notas[], int tamanho)`, que soma todas as notas num `for` e devolve a média.
2. No `main`, declare `notas[4]`, leia os 4 valores e chame a função, mostrando o resultado.

EXEMPLO DE EXECUÇÃO

```
Nota 1: 8
Nota 2: 7.5
Nota 3: 9
Nota 4: 6.5
Media da turma: 7.75
```

F5.7 A Calculadora de Médias (Revisitada)

Gabarito Oficial

CÓDIGO EM C (F5.7) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
float calcularMedia(float notas[], int tamanho) {
int i;
float soma = 0;
for (i = 0; i < tamanho; i++) {
soma = soma + notas[i];
}
return soma / tamanho;
}
int main() {
float notas[4];
int i;
for (i = 0; i < 4; i++) {
printf("Nota %d: ", i + 1);
scanf("%f", ¬as[i]);
}
printf("Media da turma: %.2f\n", calcularMedia(notas, 4));
return 0;
}
```

EXECUÇÃO REAL

```
Nota 1: 8
Nota 2: 7.5
Nota 3: 9
Nota 4: 6.5
Media da turma: 7.75
```

🧠 **Comentários & Ponto Pedagógico:**
Vale comparar diretamente com a solução da Fase 4 (que recebia nota1, nota2, nota3 separadas): aqui a função funciona para qualquer tamanho de vetor, bastando mudar o parâmetro tamanho na chamada — uma vantagem real de usar vetores.

F5.8 O Relatório do Depósito

Vetor + procedimento

O gerente de um depósito quer um relatório organizado mostrando quantas caixas há em cada uma das 6 prateleiras — e a exibição deve ficar separada da leitura, num procedimento próprio (como na Fase 4).

> 💡 **Lembrete teórico:** Assim como funções podem receber vetores, procedimentos (`void`) também podem: `void exibirEstoque(int estoque[], int tamanho)`.

Sua missão:

1. Crie o procedimento `void exibirEstoque(int estoque[], int tamanho)`, que mostra `Prateleira <i>: <valor> caixas` para cada posição.
2. No `main`, declare `estoque[6]`, leia os valores e chame o procedimento para mostrar o relatório.

EXEMPLO DE EXECUÇÃO

```
Caixas na prateleira 0: 12
Caixas na prateleira 1: 5
Caixas na prateleira 2: 0
Caixas na prateleira 3: 8
Caixas na prateleira 4: 20
Caixas na prateleira 5: 3
--- Relatorio do deposito ---
Prateleira 0: 12 caixas
Prateleira 1: 5 caixas
Prateleira 2: 0 caixas
Prateleira 3: 8 caixas
Prateleira 4: 20 caixas
Prateleira 5: 3 caixas
```

F5.8 O Relatório do Depósito

Gabarito Oficial

CÓDIGO EM C (F5.8) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
void exibirEstoque(int estoque[], int tamanho) {
int i;
for (i = 0; i < tamanho; i++) {
printf("Prateleira %d: %d caixas\n", i, estoque[i]);
}
}
int main() {
int estoque[6];
int i;
for (i = 0; i < 6; i++) {
printf("Caixas na prateleira %d: ", i);
scanf("%d", &estoque[i]);
}
printf("--- Relatorio do deposito ---\n");
exibirEstoque(estoque, 6);
return 0;
}
```

EXECUÇÃO REAL

```
Caixas na prateleira 0: 12
Caixas na prateleira 1: 5
Caixas na prateleira 2: 0
Caixas na prateleira 3: 8
Caixas na prateleira 4: 20
Caixas na prateleira 5: 3
--- Relatorio do deposito --Prateleira 0: 12 caixas
Prateleira 1: 5 caixas
Prateleira 2: 0 caixas
Prateleira 3: 8 caixas
Prateleira 4: 20 caixas
Prateleira 5: 3 caixas
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: esquecer o parâmetro tamanho no procedimento e tentar "adivinhar" o tamanho do vetor dentro dele — em C, um vetor passado como parâmetro não carrega consigo a informação do seu tamanho, por isso ele sempre precisa ser passado à parte.

Try, Catch & Learn

F5.9 O Tabuleiro Misterioso

Matriz 2×2 + soma total

Um jogo de tabuleiro simples tem uma grade de 2 linhas por 2 colunas, e cada casa guarda um valor digitado pelo jogador. O sistema precisa somar todos os valores do tabuleiro.

> 💡 **Lembrete teórico:** Para preencher uma matriz, usamos dois `for` encadeados (linha e coluna) com `scanf("%d", &matriz[i][j]);`, somando a cada leitura.

Sua missão:

1. Declare a matriz `tabuleiro[2][2]`.
2. Com dois `for` aninhados, leia o valor de cada casa e some num acumulador.
3. Mostre a soma de todas as casas do tabuleiro.

EXEMPLO DE EXECUÇÃO

```
═══ TABULEIRO 2x2 ═══
Valor em [0][0]: 4
Valor em [0][1]: 7
Valor em [1][0]: 2
Valor em [1][1]: 9
Soma de todas as casas: 22
```

F5.9 O Tabuleiro Misterioso

Gabarito Oficial

CÓDIGO EM C (F5.9) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int tabuleiro[2][2];
int i, j, soma = 0;
printf("═══ TABULEIRO 2x2 ═══\n");
for (i = 0; i < 2; i++) {
for (j = 0; j < 2; j++) {
printf("Valor em [%d][%d]: ", i, j);
scanf("%d", &tabuleiro[i][j]);
soma = soma + tabuleiro[i][j];
}
}
printf("Soma de todas as casas: %d\n", soma);
return 0;
}
```

EXECUÇÃO REAL

```
═══ TABULEIRO 2x2 ═══
Valor em [0][0]: 4
Valor em [0][1]: 7
Valor em [1][0]: 2
Valor em [1][1]: 9
Soma de todas as casas: 22
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: trocar a ordem dos índices e escrever matriz[j][i] em vez de matriz[i][j], o que aqui não altera o resultado (matriz é 2×2 e a soma é comutativa), mas quebraria em exercícios futuros que dependem da posição exata de cada valor.

Try, Catch & Learn

F5.10 O Mapa do Cinema

Matriz pré-preenchida + contagem condicional

Uma sala de cinema pequena tem 2 fileiras com 3 assentos cada. O valor 1 significa "ocupado" e 0 significa "livre". O sistema precisa desenhar o mapa da sala e contar quantos assentos estão ocupados e quantos estão livres.

> 💡 **Lembrete teórico:** Dentro do laço duplo que percorre a matriz, um `if` decide o que fazer com cada valor — exatamente como combinar decisão (Fase 2) com repetição (Fase 3), agora dentro de uma matriz.

Sua missão:

1. Declare a matriz `assentos[2][3]` já preenchida com `{{1, 0, 1}, {0, 0, 1}};`.
2. Com dois `for` aninhados, mostre `[X]` para assento ocupado e `[ ]` para livre, pulando linha ao final de cada fileira.
3. Conte e mostre o total de assentos ocupados e livres.

EXEMPLO DE EXECUÇÃO

```
=== MAPA DO CINEMA ===
[X] [ ] [X]
[ ] [ ] [X]
Assentos ocupados: 3
Assentos livres: 3
```

F5.10 O Mapa do Cinema

Gabarito Oficial

CÓDIGO EM C (F5.10) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int assentos[2][3] = {
{1, 0, 1},
{0, 0, 1}
};
int i, j, ocupados = 0, livres = 0;
printf("=== MAPA DO CINEMA ===\n");
for (i = 0; i < 2; i++) {
for (j = 0; j < 3; j++) {
if (assentos[i][j] == 1) {
printf("[X] ");
ocupados++;
} else {
printf("[ ] ");
livres++;
}
}
printf("\n");
}
printf("Assentos ocupados: %d\n", ocupados);
printf("Assentos livres: %d\n", livres);
return 0;
}
```

EXECUÇÃO REAL

```
=== MAPA DO CINEMA ===
[X] [ ] [X]
[ ] [ ] [X]
Assentos ocupados: 3
Assentos livres: 3
```

🧠 **Comentários & Ponto Pedagógico:**
Reforce que a matriz pré-preenchida {{1, 0, 1}, {0, 0, 1}} já declara as duas linhas de uma vez — cada {} interno é uma linha. Esse formato aparece bastante em exercícios que não pedem leitura do usuário. ☰6 FASE 6 — Gabarito das Questões Struct: agrupando informações relacionadas

Try, Catch & Learn

F6.1 A Ficha do Livro

Sua primeira struct

Uma biblioteca quer digitalizar o catálogo. Cada livro tem título, ano de publicação e preço, e essas três informações pertencem à mesma ficha.

> 💡 **Lembrete teórico:** Defina `struct Livro { char titulo[80]; int ano; float preco; };` antes do `main`. Para o campo de texto, use `strcpy` (lembre do `#include <string.h>`).

Sua missão:

1. Defina a struct `Livro` com os campos `titulo`, `ano` e `preco`.
2. No `main`, declare uma variável `livro1`, preencha os três campos (escolha os valores) e mostre tudo na tela.

EXEMPLO DE EXECUÇÃO

```
Titulo: O Pequeno Programador
Ano: 2021
Preco: R$ 39.90
```

F6.1 A Ficha do Livro

Gabarito Oficial

CÓDIGO EM C (F6.1) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Livro {
char titulo[80];
int ano;
float preco;
};
int main() {
struct Livro livro1;
strcpy(livro1.titulo, "O Pequeno Programador");
livro1.ano = 2021;
livro1.preco = 39.90;
printf("Titulo: %s\n", livro1.titulo);
printf("Ano: %d\n", livro1.ano);
printf("Preco: R$ %.2f\n", livro1.preco);
return 0;
}
```

EXECUÇÃO REAL

```
Titulo: O Pequeno Programador
Ano: 2021
Preco: R$ 39.90
```

> 🧠 **Comentários & Ponto Pedagógico:**
> Erro comum: tentar livro1.titulo = "..."; diretamente — um array de char não aceita atribuição com = depois da declaração; é obrigatório usar strcpy.

F6.2 A Ficha do Pet

Dois campos de texto na mesma struct

Uma clínica veterinária quer uma ficha simples para cada animal: nome, espécie e idade.

> 💡 **Lembrete teórico:** Uma struct pode ter mais de um campo de texto. Cada um precisa do seu próprio `strcpy`.

Sua missão:

1. Defina a struct `Animal` com os campos `nome`, `especie` (ambos texto) e `idade` (inteiro).
2. Preencha os dados de um pet (à sua escolha) e mostre a ficha completa.

EXEMPLO DE EXECUÇÃO

```
=== FICHA DO PET ===
Nome: Rex
Especie: Cachorro
Idade: 3 ano(s)
```

F6.2 A Ficha do Pet

Gabarito Oficial

CÓDIGO EM C (F6.2) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Animal {
char nome[30];
char especie[30];
int idade;
};
int main() {
struct Animal pet1;
strcpy(pet1.nome, "Rex");
strcpy(pet1.especie, "Cachorro");
pet1.idade = 3;
printf("=== FICHA DO PET ===\n");
printf("Nome: %s\n", pet1.nome);
printf("Especie: %s\n", pet1.especie);
printf("Idade: %d ano(s)\n", pet1.idade);
return 0;
}
```

EXECUÇÃO REAL

```
=== FICHA DO PET ===
Nome: Rex
Especie: Cachorro
Idade: 3 ano(s)
```

🧠 **Comentários & Ponto Pedagógico:**
Reforce que cada campo de texto precisa de seu próprio strcpy — é comum o aluno esquecer de copiar o segundo campo depois de já ter feito o primeiro.

F6.3 O Caixa Registradora

struct lida do usuário + cálculo

Um pequeno comércio quer registrar a venda de um produto: nome, preço unitário e quantidade — e calcular o valor total da compra.

> 💡 **Lembrete teórico:** Para ler um texto com espaços (como "Caderno Grande") diretamente do usuário dentro de um campo de struct, use `scanf(" %[^\n]", item.nome);` em vez de `strcpy`.

Sua missão:

1. Defina a struct `Produto` com os campos `nome`, `preco` e `quantidade`.
2. Leia os três valores do usuário.
3. Calcule o total (`preco * quantidade`) e mostre a nota da compra.

EXEMPLO DE EXECUÇÃO

```
Nome do produto: Caderno
Preco unitario: 15.50
Quantidade: 3
--- Nota da compra ---
Caderno x3 = R$ 46.50
```

F6.3 O Caixa Registradora

Gabarito Oficial

CÓDIGO EM C (F6.3) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Produto {
char nome[50];
float preco;
int quantidade;
};
int main() {
struct Produto item;
float total;
printf("Nome do produto: ");
scanf(" %[^\n]", item.nome);
printf("Preco unitario: ");
scanf("%f", &item.preco);
printf("Quantidade: ");
scanf("%d", &item.quantidade);
total = item.preco * item.quantidade;
printf("--- Nota da compra ---\n");
printf("%s x%d = R$ %.2f\n", item.nome, item.quantidade, total);
return 0;
}
```

EXECUÇÃO REAL

```
Nome do produto: Caderno
Preco unitario: 15.50
Quantidade: 3
--- Nota da compra --Caderno x3 = R$ 46.50
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: usar scanf("%s", item.nome) para ler o nome — isso funciona para uma palavra só, mas corta no primeiro espaço ("Caderno Grande" viraria só "Caderno"). O formato " %[^\n]" lê a linha inteira, inclusive espaços.

Try, Catch & Learn

F6.4 O Boletim da Turma

Vetor de struct

Uma professora quer um boletim para 3 alunos da turma, cada um com nome e duas notas, calculando a média de cada um no final.

> 💡 **Lembrete teórico:** Um vetor de struct se declara como `struct Aluno turma[3];`. Cada posição `turma[i]` é uma ficha completa, acessada normalmente com `turma[i].nome`, `turma[i].nota1`...

Sua missão:

1. Defina a struct `Aluno` com `nome`, `nota1` e `nota2`.
2. Declare o vetor `turma[3]` e, com um `for`, leia nome e as duas notas de cada aluno.
3. Com outro `for`, calcule e mostre a média de cada aluno.

EXEMPLO DE EXECUÇÃO

```
--- Aluno 1 ---
Nome: Ana
Nota 1: 8
Nota 2: 7
--- Aluno 2 ---
Nome: Bruno
Nota 1: 5
Nota 2: 6
--- Aluno 3 ---
Nome: Carla
Nota 1: 9
Nota 2: 10
=== BOLETIM ===
Ana - Media: 7.5
Bruno - Media: 5.5
Carla - Media: 9.5
```

F6.4 O Boletim da Turma

Gabarito Oficial

CÓDIGO EM C (F6.4) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Aluno {
char nome[50];
float nota1;
float nota2;
};
int main() {
struct Aluno turma[3];
int i;
float media;
for (i = 0; i < 3; i++) {
printf("--- Aluno %d ---\n", i + 1);
printf("Nome: ");
scanf(" %[^\n]", turma[i].nome);
printf("Nota 1: ");
scanf("%f", &turma[i].nota1);
printf("Nota 2: ");
scanf("%f", &turma[i].nota2);
}
printf("\n=== BOLETIM ===\n");
for (i = 0; i < 3; i++) {
media = (turma[i].nota1 + turma[i].nota2) / 2;
printf("%s - Media: %.1f\n", turma[i].nome, media);
}
return 0;
}
```

EXECUÇÃO REAL

```
--- Aluno 1 --Nome: Ana
Nota 1: 8
Nota 2: 7
--- Aluno 2 --Nome: Bruno
Nota 1: 5
Nota 2: 6
--- Aluno 3 --Nome: Carla
Nota 1: 9
Nota 2: 10
=== BOLETIM ===
Ana - Media: 7.5
Bruno - Media: 5.5
Carla - Media: 9.5
```

🧠 **Comentários & Ponto Pedagógico:**
Mostre a diferença entre turma[i].nome (correto: acessa o campo nome da ficha i) e o erro comum turma.nome[i] (que não compila, pois turma sozinho não é uma struct, é um vetor delas).

F6.5 O Mapa do Tesouro

struct simples + função que recebe struct

Um mapa de tesouro marca pontos importantes com coordenadas x e y. Vamos criar uma struct Ponto e uma função que mostra qualquer ponto de forma padronizada.

> 💡 **Lembrete teórico:** Uma struct pode ser passada como parâmetro de uma função, assim como int ou float: `void mostrarPonto(struct Ponto p)`.

Sua missão:

1. Defina a struct `Ponto` com os campos x e y (inteiros).
2. Crie o procedimento `void mostrarPonto(struct Ponto p)`, que mostra `(x, y)`.
3. No `main`, crie dois pontos (origem e destino) com valores à sua escolha e mostre os dois usando o procedimento.

EXEMPLO DE EXECUÇÃO

```
Ponto de origem: (0, 0)
Ponto de destino: (5, 8)
```

F6.5 O Mapa do Tesouro

Gabarito Oficial

CÓDIGO EM C (F6.5) GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Ponto {
int x;
int y;
};
void mostrarPonto(struct Ponto p) {
printf("(%d, %d)\n", p.x, p.y);
}
int main() {
struct Ponto origem;
struct Ponto destino;
origem.x = 0;
origem.y = 0;
destino.x = 5;
destino.y = 8;
printf("Ponto de origem: ");
mostrarPonto(origem);
printf("Ponto de destino: ");
mostrarPonto(destino);
return 0;
}
```

EXECUÇÃO REAL

```
Ponto de origem: (0, 0)
Ponto de destino: (5, 8)
```

🧠 **Comentários & Ponto Pedagógico:**
Aqui a struct é passada por cópia (sem *), porque o procedimento só precisa ler os valores para mostrá-los, não alterar o ponto original. Compare com a Questão 7, que exige ponteiro justamente porque precisa alterar.

Try, Catch & Learn

F6.6 A Garagem de Carros Antigos

Vetor de struct + procedimento

Um colecionador quer catalogar 3 carros de sua garagem: modelo, ano e preço, exibidos de forma padronizada.

> 💡 **Lembrete teórico:** Combine o que já viu: um vetor de struct (Questão 4) com um procedimento que recebe uma struct (Questão 5).

Sua missão:

1. Defina a struct `Carro` com `modelo`, `ano` e `preco`.
2. Declare o vetor `garagem[3]` e preencha os três carros diretamente no código (sem scanf, à sua escolha).
3. Crie o procedimento `void mostrarCarro(struct Carro c)` e use-o num `for` para exibir a garagem inteira.

EXEMPLO DE EXECUÇÃO

```
═══ GARAGEM ═══
Fusca (1978) - R$ 25000.00
Civic (2020) - R$ 98000.00
Gol (2015) - R$ 42000.00
```

F6.6 A Garagem de Carros Antigos

Gabarito Oficial

CÓDIGO EM C (F6.6) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Carro {
char modelo[40];
int ano;
float preco;
};
void mostrarCarro(struct Carro c) {
printf("%s (%d) - R$ %.2f\n", c.modelo, c.ano, c.preco);
}
int main() {
struct Carro garagem[3];
int i;
strcpy(garagem[0].modelo, "Fusca");
garagem[0].ano = 1978;
garagem[0].preco = 25000.00;
strcpy(garagem[1].modelo, "Civic");
garagem[1].ano = 2020;
garagem[1].preco = 98000.00;
strcpy(garagem[2].modelo, "Gol");
garagem[2].ano = 2015;
garagem[2].preco = 42000.00;
printf("=== GARAGEM ===\n");
for (i = 0; i < 3; i++) {
mostrarCarro(garagem[i]);
}
return 0;
}
```

EXECUÇÃO REAL

```
=== GARAGEM ===
Fusca (1978) - R$ 25000.00
Civic (2020) - R$ 98000.00
Gol (2015) - R$ 42000.00
```

🧠 **Comentários & Ponto Pedagógico:**
Combina os dois padrões já vistos (vetor de struct da Q4, struct em função da Q5). Vale perguntar: por que mostrarCarro recebe só uma struct (struct Carro c), e não o vetor inteiro? Porque cada chamada, dentro do for do main, cuida de um carro por vez.

F6.7 O Duelo de Treinamento

struct modificada por uma função (ponteiro)

Um jogo de RPG precisa que a função de ataque altere de verdade a vida do personagem, e não apenas uma cópia dela. Para isso, vamos usar um conceito novo: passar o endereço da struct para a função, usando * e &.

> 💡 **Lembrete teórico:** Até aqui, toda struct passada para uma função era uma **cópia** — alterar os campos dentro da função não mudava o original. Para alterar de verdade, a função recebe um **ponteiro** para a struct (`struct Jogador *alvo`) e acessa os campos com `->` em vez de `.`. A chamada usa `&` para passar o endereço: `atacar(&heroi, 30);`.

Sua missão:

1. Defina a struct `Jogador` com `nome`, `vida` e `ataque`.
2. Crie o procedimento `void atacar(struct Jogador *alvo, int dano)`, que subtrai `dano` de `alvo->vida`.
3. No `main`, crie um jogador com 100 de vida, ataque-o duas vezes com valores diferentes de dano e mostre a vida atual após cada ataque.

EXEMPLO DE EXECUÇÃO

```
Cavaleiro comeca com 100 de vida.
Cavaleiro foi atacado! Vida atual: 70
Cavaleiro foi atacado de novo! Vida atual: 50
```

> 🔍 **Por que isso funciona?:** `alvo` guarda o endereço de memória de `heroi`, não uma cópia. Por isso, `alvo->vida = alvo->vida - dano;` altera a variável original no `main`. É a mesma ideia por trás do `&idade` que você já usa no `scanf` há várias fases — agora aplicada a uma struct inteira.

F6.7 O Duelo de Treinamento

Gabarito Oficial

CÓDIGO EM C (F6.7) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Jogador {
char nome[30];
int vida;
int ataque;
};
void atacar(struct Jogador *alvo, int dano) {
alvo->vida = alvo->vida - dano;
}
int main() {
struct Jogador heroi;
strcpy(heroi.nome, "Cavaleiro");
heroi.vida = 100;
heroi.ataque = 15;
printf("%s comeca com %d de vida.\n", heroi.nome, heroi.vida);
atacar(&heroi, 30);
printf("%s foi atacado! Vida atual: %d\n", heroi.nome, heroi.vida);
atacar(&heroi, 20);
printf("%s foi atacado de novo! Vida atual: %d\n", heroi.nome, heroi.vida);
return 0;
}
```

EXECUÇÃO REAL

```
Cavaleiro comeca com 100 de vida.
Cavaleiro foi atacado! Vida atual: 70
Cavaleiro foi atacado de novo! Vida atual: 50
```

🧠 **Comentários & Ponto Pedagógico:**

Ponto pedagógico central desta fase: sem o ponteiro, atacar receberia uma cópia de heroi, e alvo.vida = alvo.vida - dano; alteraria só essa cópia — a vida de heroi no main continuaria 100 para sempre. O ponteiro (*alvo, chamado com &heroi) é o que permite a alteração "vazar" de volta para a variável original. Erro comum: esquecer o & na chamada (atacar(heroi, 30);), o que não compila, já que a função espera um endereço (struct Jogador *) e não a struct em si.

F6.8 O Cálculo do Bônus

Função com struct que devolve valor

O setor de RH de uma empresa quer calcular automaticamente o bônus de um funcionário: 10% do salário.

> 💡 **Lembrete teórico:** Uma função pode receber uma struct e devolver um valor comum (`float`), sem precisar alterar a struct original — nesse caso, não precisamos de ponteiro, só de ler os campos.

Sua missão:

1. Defina a struct `Funcionario` com `nome` e `salario`.
2. Crie a função `float calcularBonus(struct Funcionario f)`, que devolve `f.salario * 0.10`.
3. No `main`, leia os dados de um funcionário, calcule o bônus com a função e mostre o resultado.

EXEMPLO DE EXECUÇÃO

```
Nome do funcionario: Mariana
Salario: 2500
Mariana recebera um bonus de R$ 250.00
```

F6.8 O Cálculo do Bônus

Gabarito Oficial

CÓDIGO EM C (F6.8) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Funcionario {
char nome[50];
float salario;
};
float calcularBonus(struct Funcionario f) {
return f.salario * 0.10;
}
int main() {
struct Funcionario pessoa;
float bonus;
printf("Nome do funcionario: ");
scanf(" %[^\n]", pessoa.nome);
printf("Salario: ");
scanf("%f", &pessoa.salario);
bonus = calcularBonus(pessoa);
printf("%s recebera um bonus de R$ %.2f\n", pessoa.nome, bonus);
return 0;
}
```

EXECUÇÃO REAL

```
Nome do funcionario: Mariana
Salario: 2500
Mariana recebera um bonus de R$ 250.00
```

🧠 **Comentários & Ponto Pedagógico:**
Diferente da Q7, aqui não precisamos de ponteiro: a função só lê o salário para calcular e devolver um número novo, sem alterar a struct Funcionario original.

Try, Catch & Learn

F6.9 A Nota Fiscal do Mercado

Vetor de struct + total acumulado

Um mini mercado quer emitir uma nota fiscal simples para 4 produtos do carrinho, somando o total da compra.

> 💡 **Lembrete teórico:** Combine tudo que já viu: vetor de struct, percorrido com `for`, acumulando um total — o mesmo padrão da Questão 3 da Fase 5, agora com cada item sendo uma ficha completa em vez de um número solto.

Sua missão:

1. Defina a struct `Produto` com `nome` e `preco`.
2. Declare o vetor `carrinho[4]` e preencha os 4 produtos diretamente no código.
3. Com um `for`, mostre cada item (nome e preço) e some o total.
4. Mostre o total da nota fiscal.

EXEMPLO DE EXECUÇÃO

```
=== NOTA FISCAL ===
Arroz      R$ 22.50
Feijao     R$ 8.90
Leite      R$ 4.50
Pao        R$ 7.20
-------------------------
TOTAL: R$ 43.10
```

F6.9 A Nota Fiscal do Mercado

Gabarito Oficial

CÓDIGO EM C (F6.9) GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Produto {
char nome[40];
float preco;
};
int main() {
struct Produto carrinho[4];
int i;
float total = 0;
strcpy(carrinho[0].nome, "Arroz");
carrinho[0].preco = 22.50;
strcpy(carrinho[1].nome, "Feijao");
carrinho[1].preco = 8.90;
strcpy(carrinho[2].nome, "Leite");
carrinho[2].preco = 4.50;
strcpy(carrinho[3].nome, "Pao");
carrinho[3].preco = 7.20;
printf("═══ NOTA FISCAL ═══\n");
for (i = 0; i < 4; i++) {
printf("%-10s R$ %.2f\n", carrinho[i].nome, carrinho[i].preco);
total = total + carrinho[i].preco;
}
printf("-------------------------\n");
printf("TOTAL: R$ %.2f\n", total);
return 0;
}
```

EXECUÇÃO REAL

```
═══ NOTA FISCAL ═══
Arroz
R$ 22.50
Feijao
R$ 8.90
Leite
R$ 4.50
Pao
R$ 7.20
-------------------------TOTAL: R$ 43.10
```

🧠 **Comentários & Ponto Pedagógico:**
O acumulador total soma um campo da struct (carrinho[i].preco) a cada volta — mesma lógica da Questão 3 da Fase 5, só que agora dentro de um vetor de fichas em vez de um vetor de números soltos.

Try, Catch & Learn

F6.10 A Apuração da Eleição

Vetor de struct + busca do maior

Uma eleição estudantil tem 3 candidatos. O sistema precisa ler os votos de cada um e anunciar o vencedor.

> 💡 **Lembrete teórico:** Combine a busca do maior valor (Questão 4 da Fase 5) com struct: em vez de guardar só o maior valor, guarde o índice de quem está vencendo (`maiorIndice`), para poder mostrar o nome do candidato no final.

Sua missão:

1. Defina a struct `Candidato` com `nome` e `votos`.
2. Declare o vetor `eleicao[3]`, já com os nomes preenchidos no código, e leia os votos de cada um.
3. Encontre o índice do candidato com mais votos (mesmo padrão de "maior valor", mas guardando a posição).
4. Mostre o nome e os votos do(a) vencedor(a).

EXEMPLO DE EXECUÇÃO

```
Votos para Ana: 120
Votos para Bruno: 340
Votos para Carla: 210
=== RESULTADO ===
Vencedor(a): Bruno com 340 votos!
```

F6.10 A Apuração da Eleição

Gabarito Oficial

CÓDIGO EM C (F6.10) GCC -WALL -WEXTRA

```c
#include <stdio.h>
#include <string.h>
struct Candidato {
char nome[40];
int votos;
};
int main() {
struct Candidato eleicao[3];
int i, maiorIndice;
strcpy(eleicao[0].nome, "Ana");
strcpy(eleicao[1].nome, "Bruno");
strcpy(eleicao[2].nome, "Carla");
for (i = 0; i < 3; i++) {
printf("Votos para %s: ", eleicao[i].nome);
scanf("%d", &eleicao[i].votos);
}
maiorIndice = 0;
for (i = 1; i < 3; i++) {
if (eleicao[i].votos > eleicao[maiorIndice].votos) {
maiorIndice = i;
}
}
printf("\n=== RESULTADO ===\n");
printf("Vencedor(a): %s com %d votos!\n", eleicao[maiorIndice].nome, eleicao[maiorIndi
return 0;
}
```

EXECUÇÃO REAL

```
Votos para Ana: 120
Votos para Bruno: 340
Votos para Carla: 210
=== RESULTADO ===
Vencedor(a): Bruno com 340 votos!
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: guardar o maior número de votos numa variável separada (int maiorVotos) em vez do índice. Funciona para mostrar a contagem, mas complica mostrar o nome do vencedor depois — guardar o índice (maiorIndice) resolve as duas coisas de uma vez, pois dá acesso à ficha inteira.

D1 A Caravana de Mercadores

Desafio 1 · Vetor + estatísticas completas

Uma caravana carrega 10 itens de pesos diferentes. O chefe da caravana precisa de um relatório completo: peso total, peso médio, o item mais pesado, o mais leve, e um alerta se o peso total ultrapassar o limite seguro de 500 kg.

Sua missão:

1. Declare o vetor `caravana[10]` e leia os 10 pesos.
2. Num único `for`, calcule a soma, e também atualize o maior e o menor peso encontrados (sem usar laços separados para cada estatística).
3. Calcule a média (atenção: para um resultado com casas decimais, converta a soma para `float` antes de dividir, usando `(float) soma / 10`).
4. Mostre o relatório completo e o alerta, se aplicável.

EXEMPLO DE EXECUÇÃO

```
=== Exemplo: caravana acima do limite ===
Peso do item 1 (kg): 80
Peso do item 2 (kg): 45
Peso do item 3 (kg): 60
Peso do item 4 (kg): 30
Peso do item 5 (kg): 55
Peso do item 6 (kg): 40
Peso do item 7 (kg): 70
Peso do item 8 (kg): 25
Peso do item 9 (kg): 50
Peso do item 10 (kg): 65
--- Relatorio da caravana ---
Peso total: 520 kg
Peso medio: 52.00 kg
Item mais pesado: 80 kg
Item mais leve: 25 kg
ALERTA: caravana acima do limite de 500 kg!

=== Exemplo: caravana dentro do limite ===
Peso do item 1 (kg): 10
Peso do item 2 (kg): 20
Peso do item 3 (kg): 15
Peso do item 4 (kg): 25
Peso do item 5 (kg): 30
Peso do item 6 (kg): 5
Peso do item 7 (kg): 18
Peso do item 8 (kg): 22
Peso do item 9 (kg): 12
Peso do item 10 (kg): 8
--- Relatorio da caravana ---
Peso total: 165 kg
Peso medio: 16.50 kg
Item mais pesado: 30 kg
Item mais leve: 5 kg
Peso dentro do limite seguro.
```

💡 **Pista:** `(float) soma / 10` é um cast: converte `soma` para decimal só para esta conta, sem mudar o tipo da variável `soma` em si. Sem o cast, `soma / 10` faria divisão inteira e perderia as casas decimais.

D1 A Caravana de Mercadores

Gabarito Oficial

CÓDIGO EM C (D1) GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int caravana[10];
int i, soma = 0, maior, menor;
float media;
for (i = 0; i < 10; i++) {
printf("Peso do item %d (kg): ", i + 1);
scanf("%d", &caravana[i]);
}
maior = caravana[0];
menor = caravana[0];
for (i = 0; i < 10; i++) {
soma = soma + caravana[i];
if (caravana[i] > maior) {
maior = caravana[i];
}
if (caravana[i] < menor) {
menor = caravana[i];
}
}
media = (float) soma / 10;
printf("--- Relatorio da caravana ---\n");
printf("Peso total: %d kg\n", soma);
printf("Peso medio: %.2f kg\n", media);
printf("Item mais pesado: %d kg\n", maior);
printf("Item mais leve: %d kg\n", menor);
if (soma > 500) {
printf("ALERTA: caravana acima do limite de 500 kg!\n");
} else {
printf("Peso dentro do limite seguro.\n");
}
return 0;
}
```

EXECUÇÃO REAL

```
: acima do limite
Peso do item 1 (kg): 80
Peso do item 2 (kg): 45
Peso do item 3 (kg): 60
Peso do item 4 (kg): 30
Peso do item 5 (kg): 55
Peso do item 6 (kg): 40
Peso do item 7 (kg): 70
Peso do item 8 (kg): 25
Peso do item 9 (kg): 50
Peso do item 10 (kg): 65
--- Relatorio da caravana --Peso total: 520 kg
Peso medio: 52.00 kg
Item mais pesado: 80 kg
Item mais leve: 25 kg
ALERTA: caravana acima do limite de 500 kg!



Execução: dentro do limite
Peso do item 1 (kg): 10
Peso do item 2 (kg): 20
Peso do item 3 (kg): 15
Peso do item 4 (kg): 25
Peso do item 5 (kg): 30
Peso do item 6 (kg): 5
Peso do item 7 (kg): 18
Peso do item 8 (kg): 22
Peso do item 9 (kg): 12
Peso do item 10 (kg): 8
--- Relatorio da caravana --Peso total: 165 kg
Peso medio: 16.50 kg
Item mais pesado: 30 kg
Item mais leve: 5 kg
Peso dentro do limite seguro.
```

🧠 **Comentários & Ponto Pedagógico:**
Ponto pedagógico central: calcular quatro estatísticas (soma, maior, menor e, depois, a média) num único for, em vez de quatro laços separados — mostra que várias atualizações podem acontecer na mesma volta da repetição. Erro comum: calcular media = soma / 10; sem o cast (float), o que faz uma divisão inteira e descarta a parte decimal (por exemplo, 52.00 viraria 52, mas 51.50 arredondaria para baixo, perdendo precisão).

D2 O Jogo da Velha Matemático

Desafio 2 · Matriz 3×3 + funções + diagonais

Um professor de matemática quer um "jogo da velha" especial: em vez de X e O, o tabuleiro 3×3 é preenchido com números, e o programa calcula a soma de cada diagonal.

> 💡 **Lembrete teórico:** A diagonal principal de uma matriz quadrada é onde `linha == coluna` (`m[i][i]`). A diagonal secundária, para uma matriz 3×3, é onde a soma dos índices dá 2 (`m[i][2-i]`) — o mesmo princípio do Desafio 1 da Fase 3, agora aplicado a somas em vez de desenho.

Sua missão:

1. Crie o procedimento `void preencherMatriz(int m[3][3])`, que lê os 9 valores com dois `for` aninhados.
2. Crie a função `int somarDiagonalPrincipal(int m[3][3])`, que soma `m[i][i]` para `i` de 0 a 2.
3. Crie a função `int somarDiagonalSecundaria(int m[3][3])`, que soma `m[i][2-i]`.
4. Crie o procedimento `void exibirMatriz(int m[3][3])`, que mostra o tabuleiro formatado.
5. No `main`, declare a matriz, preencha, exiba e mostre as duas somas.

EXEMPLO DE EXECUÇÃO

```
=== PREENCHENDO O TABULEIRO 3x3 ===
Valor da casa [0][0]: 1
Valor da casa [0][1]: 2
Valor da casa [0][2]: 3
Valor da casa [1][0]: 4
Valor da casa [1][1]: 5
Valor da casa [1][2]: 6
Valor da casa [2][0]: 7
Valor da casa [2][1]: 8
Valor da casa [2][2]: 9
--- Tabuleiro ---
 1  2  3
 4  5  6
 7  8  9
Soma da diagonal principal: 15
Soma da diagonal secundaria: 15
```

D2 O Jogo da Velha Matemático

Gabarito Oficial

CÓDIGO EM C (D2) GCC -WALL -WEXTRA

```c
#include <stdio.h>
void preencherMatriz(int m[3][3]) {
int i, j;
for (i = 0; i < 3; i++) {
for (j = 0; j < 3; j++) {
printf("Valor da casa [%d][%d]: ", i, j);
scanf("%d", &m[i][j]);
}
}
}
int somarDiagonalPrincipal(int m[3][3]) {
int i, soma = 0;
for (i = 0; i < 3; i++) {
soma = soma + m[i][i];
}
return soma;
}
int somarDiagonalSecundaria(int m[3][3]) {
int i, soma = 0;
for (i = 0; i < 3; i++) {
soma = soma + m[i][2 - i];
}
return soma;
}
void exibirMatriz(int m[3][3]) {
int i, j;
for (i = 0; i < 3; i++) {
for (j = 0; j < 3; j++) {
printf("%4d", m[i][j]);
}
printf("\n");
}
}
int main() {
int jogoDaVelha[3][3];
printf("=== PREENCHENDO O TABULEIRO 3x3 ===\n");
preencherMatriz(jogoDaVelha);
printf("\n--- Tabuleiro ---\n");
exibirMatriz(jogoDaVelha);
printf("\nSoma da diagonal principal: %d\n", somarDiagonalPrincipal(jogoDaVelha));
printf("Soma da diagonal secundaria: %d\n", somarDiagonalSecundaria(jogoDaVelha));
return 0;
}
```

EXECUÇÃO REAL

```
=== PREENCHENDO O TABULEIRO 3x3 ===
Valor da casa [0][0]: 1
Valor da casa [0][1]: 2
Valor da casa [0][2]: 3
Valor da casa [1][0]: 4
Valor da casa [1][1]: 5
Valor da casa [1][2]: 6
Valor da casa [2][0]: 7
Valor da casa [2][1]: 8
Valor da casa [2][2]: 9
--- Tabuleiro --1
2
3
4
5
6
7
8
9
Soma da diagonal principal: 15
Soma da diagonal secundaria: 15
```

🧠 **Comentários & Ponto Pedagógico:**
Erro comum: confundir a fórmula da diagonal secundária. Para uma matriz 3×3 (índices de 0 a 2), a diagonal secundária é onde coluna = 2 - linha, ou seja, m[0][2], m[1][1], m[2][0]. Vale desenhar o tabuleiro no quadro para visualizar antes de programar.

D3 O Cofre de Quatro Pinos

Desafio 3 · Dois vetores comparados + laço com break

Um cofre de segurança tem uma senha secreta de 4 pinos (dígitos de 0 a 9). O visitante tem até 5 tentativas; a cada uma, o cofre informa quantos pinos acertou na posição certa (como num jogo de adivinhação).

Fonte	O que guarda
`senha[4]`	A senha secreta, fixa no código
`tentativa[4]`	O palpite do visitante em cada rodada

Sua missão:

1. Declare `senha[4]` com os valores `{3, 7, 1, 9}` (fixos no código).
2. Use um `for` de até 5 rodadas. Em cada uma: leia 4 valores no vetor `tentativa`, compare posição a posição com `senha`, contando os acertos.
3. Mostre quantos pinos acertou. Se acertar os 4, mostre `COFRE ABERTO!` e pare o laço com `break`.
4. Se as 5 tentativas acabarem sem acerto total, mostre que o cofre permanece fechado.

EXEMPLO DE EXECUÇÃO

```
=== Exemplo: acerto na 3ª tentativa ===
=== COFRE DE 4 PINOS ===
Tentativa 1 de 5
Pino 1 (0-9): 1
Pino 2 (0-9): 2
Pino 3 (0-9): 3
Pino 4 (0-9): 4
Pinos corretos na posicao certa: 0 de 4
Tentativa 2 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 5
Pinos corretos na posicao certa: 3 de 4
Tentativa 3 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 9
Pinos corretos na posicao certa: 4 de 4
COFRE ABERTO!

=== Exemplo: 5 tentativas sem acerto ===
=== COFRE DE 4 PINOS ===
Tentativa 1 de 5
Pino 1 (0-9): 0
Pino 2 (0-9): 0
Pino 3 (0-9): 0
Pino 4 (0-9): 0
Pinos corretos na posicao certa: 0 de 4
...
Tentativas esgotadas! O cofre permanece fechado.
```

> 💡 **Pista:** A comparação precisa ser **posição a posição**: `tentativa[0]` só conta como acerto se for igual a `senha[0]`, e assim por diante. Não vale comparar se o dígito existe em qualquer posição da senha, só se está no lugar certo.

D3 O Cofre de Quatro Pinos

Gabarito Oficial

CÓDIGO EM C (D3) — GCC -WALL -WEXTRA

```c
#include <stdio.h>
int main() {
int senha[4];
int tentativa[4];
int i, acertos;
int acertouTudo = 0;
int rodada;
senha[0] = 3; senha[1] = 7; senha[2] = 1; senha[3] = 9;
printf("=== COFRE DE 4 PINOS ===\n");
for (rodada = 1; rodada <= 5; rodada++) {
printf("\nTentativa %d de 5\n", rodada);
acertos = 0;
for (i = 0; i < 4; i++) {
printf("Pino %d (0-9): ", i + 1);
scanf("%d", &tentativa[i]);
}
for (i = 0; i < 4; i++) {
if (tentativa[i] == senha[i]) {
acertos++;
}
}
printf("Pinos corretos na posicao certa: %d de 4\n", acertos);
if (acertos == 4) {
printf("COFRE ABERTO!\n");
acertouTudo = 1;
break;
}
}
if (acertouTudo == 0) {
printf("\nTentativas esgotadas! O cofre permanece fechado.\n");
}
return 0;
}
```

EXECUÇÃO REAL

```
: acerto na 3ª tentativa
═══ COFRE DE 4 PINOS ═══
Tentativa 1 de 5
Pino 1 (0-9): 1
Pino 2 (0-9): 2
Pino 3 (0-9): 3
Pino 4 (0-9): 4
Pinos corretos na posicao certa: 0 de 4
Tentativa 2 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 5
Pinos corretos na posicao certa: 3 de 4
Tentativa 3 de 5
Pino 1 (0-9): 3
Pino 2 (0-9): 7
Pino 3 (0-9): 1
Pino 4 (0-9): 9
Pinos corretos na posicao certa: 4 de 4
COFRE ABERTO!

Execução: 5 tentativas sem acerto
═══ COFRE DE 4 PINOS ═══
Tentativa 1 de 5
Pino 1 (0-9): 0
Pino 2 (0-9): 0
Pino 3 (0-9): 0
Pino 4 (0-9): 0
Pinos corretos na posicao certa: 0 de 4
Tentativa 2 de 5
Pino 1 (0-9): 1
Pino 2 (0-9): 1
Pino 3 (0-9): 1
Pino 4 (0-9): 1
Pinos corretos na posicao certa: 1 de 4
Tentativa 3 de 5
Pino 1 (0-9): 2
Pino 2 (0-9): 2
Pino 3 (0-9): 2
Pino 4 (0-9): 2
Pinos corretos na posicao certa: 0 de 4
Tentativa 4 de 5
Pino 1 (0-9): 5
Pino 2 (0-9): 5
Pino 3 (0-9): 5
Pino 4 (0-9): 5
Pinos corretos na posicao certa: 0 de 4
Tentativa 5 de 5
Pino 1 (0-9): 6
Pino 2 (0-9): 6
Pino 3 (0-9): 6
Pino 4 (0-9): 6
Pinos corretos na posicao certa: 0 de 4
Tentativas esgotadas! O cofre permanece fechado.
```

🧠 **Comentários & Ponto Pedagógico:**
Ponto pedagógico central: a comparação é posição a posição (tentativa[i] == senha[i]), diferente da busca livre da Questão 6 da Fase 5 (que procurava um valor em qualquer posição do vetor). É uma boa

oportunidade para comparar os dois tipos de busca lado a lado. Erro comum: reiniciar a variável acertos fora do laço de rodadas, fazendo os acertos se acumularem entre tentativas diferentes em vez de contar certo a cada rodada nova.